Algorithm Analysis and Correctness: Solutions

Indices, logarithms, and the cost model follow the exercises. Exact examples and general arguments are distinguished below.

A01-E01 What does sorting promise?

Output	Sorted	Preserved	Stable	Reason
(1)	Yes	Yes	Yes	All records remain; b precedes d and a precedes c
(2)	Yes	Yes	No	The two records with key 1 have reversed order
(3)	Yes	No	Not a valid complete sorting	Record $1d$ is missing

An empty input has the empty sequence as its sorting. Stability restricts the relative order of equal-key records; it does not replace the requirement to preserve all records.

A01-E02 Prove first-match search

Target	Compared indices	Return	Comparisons
6	1	1	1
5	1, 2, 3, 4	4	4
9	1, 2, 3, 4	NIL	4

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Invariant: before each guard, $1 \leq i \leq n + 1$ and no entry of $A[1..i - 1]$ matches. Initially this prefix is empty. A failed comparison followed by incrementing i adds exactly the position just checked. A returned position matches, while every earlier position does not. On guard failure, $i = n + 1$, so no position matches.

Each nonreturning iteration decreases the nonnegative integer $n - i + 1$ by 1. There are at most n iterations. For an empty array, no iteration occurs and NIL is returned. Every exit satisfies the specification.

A01-E03 What does partial correctness leave unproved?

P has no terminating execution. The implication "if it terminates, the output is correct" therefore holds vacuously. Total correctness fails because it does not terminate on any legal input. Partial correctness does not imply completion.

A smallest counterexample for Q is $n = 1, A = [8], x = 8$. The initial `i < n` test is false, so it returns NIL without checking the only record. Q terminates, but violates the output specification. Restore `i <= n`. Searching only the first $n - 1$ positions would require a different problem specification.

A01-E04 Count insertion sort independently

j / key	Key comparisons	Shifts h_j	q_j	Result
2 / $2b$	$4 > 2$ true	1	0	$2b, 4a, 4c, 1d$
3 / $4c$	$4 > 4$ false	0	1	$2b, 4a, 4c, 1d$
4 / $1d$	$4c > 1, 4a > 1, 2b > 1$ true	3	0	$1d, 2b, 4a, 4c$

Total shifts: $1 + 0 + 3 = 4$. Key comparisons: $4 + (0 + 1 + 0) = 5$. Complete inner guards: $(1 + 1) + (0 + 1) + (3 + 1) = 7$. Array writes: four shift writes plus three final key writes, totaling seven. The unchanged-position write in iteration 3 still counts under this pseudocode.

Original inversions: $(4a, 2b), (4a, 1d), (2b, 1d), (4c, 1d)$, four pairs. Equal-key records do not form a strict inversion. The sorted result keeps $4a$ before $4c$.

A01-E05 Change one comparison symbol

For $[2a, 2b]$, save $2b$. The nonstrict key comparison succeeds, so $2a$ shifts right and $2b$ is placed first, producing $[2b, 2a]$.

The result is sorted and record-preserving, and execution terminates, but stability fails. There were zero strict inversions and one shift. The new shift count equals the number of original pairs $p < q$ with $S[p].key \geq S[q].key$.

Change the inner right-segment condition from "greater than key" to "greater than or equal to key." On stopping, the left segment is strictly smaller. The outer loop preserves order and records, but no longer stability.

A01-E06 Specify a distribution before averaging

Success has probability $3/4$. Each position, conditional on success, has probability $1/4$; each unconditional success event therefore has probability $3/16$.

Event	Probability	Comparisons	Probability times cost
First match at 1	$3/16$	1	$3/16$
At 2	$3/16$	2	$6/16$
At 3	$3/16$	3	$9/16$
At 4	$3/16$	4	$12/16$
Failure	$1/4$	4	1

The expectation is $30/16 + 1 = 23/8 = 2.875$. Equivalently, conditional expectation gives $(3/4)(5/2) + (1/4)4$. The expression $(n+1)/2$ concerns uniform successful positions only; it cannot absorb failure or replace an unspecified input distribution.

A01-E07 Prove a tight bound with explicit quantifiers

For $n \geq 1$, $5n^2 \leq f(n)$. Also $3n \leq 3n^2$ and $7 \leq 7n^2$, giving $f(n) \leq 15n^2$. Thus $c_1 = 5, c_2 = 15, n_0 = 1$ witness the tight bound.

Suppose $f \in O(n)$. Some fixed $c > 0, n_0$ would satisfy $f(n) \leq cn$ for every $n \geq n_0$. Choose an integer $n > \max\{n_0, 1, c/5\}$. Then $5n^2 \leq f(n) \leq cn$, implying $n \leq c/5$, a contradiction. This construction works against arbitrary candidate constants, refuting the existential quantifiers.

A01-E08 little-o and the logarithm trap

Given any $c > 0$, choose an integer $n_0 > 9/c$. Every $n \geq n_0$ then satisfies $9n < cn^2$, proving $9n \in o(n^2)$.

For $9n^2 \in o(n^2)$, the definition would have to hold for $c = 9$, requiring $9n^2 < 9n^2$, which is impossible.

Set $f = n \lg n, g = n$. Their ratio is $\lg n \to \infty$, so $f \in \omega(g)$, not $\Theta(g)$. Although $\lg f / \lg g = 1 + \lg \lg n / \lg n \to 1$, taking logs compresses an unbounded multiplicative ratio into a relatively small additive term. It does not bound the original ratio between fixed positive constants.

A01-E09 Compare oscillating functions

Function	$O(n)$	$\Omega(n)$	$\Theta(n)$	$o(n)$	$\omega(n)$
f	Yes	Yes	Yes	No	No
h	No	No	No	No	No

The ratio f/n alternates between 1 and 3, so $n \le f \le 3n$ proves a tight bound. It tends neither to zero nor to infinity.

Given any c, n_0, an even integer $n > \max(c, n_0)$ makes $h/n = n > c$, refuting an upper bound. An odd integer $n > \max(1/c, n_0)$ makes $h/n = 1/n < c$, refuting a lower bound. Since little-o implies Big-O, and little-omega implies Big-Omega, neither strict relation holds for h.

A01-E10 Three loops, three different costs

Loop	Exact expression	Tight bound	$n = 6$
(1)	$n(n+1)/2$	$\Theta(n^2)$	21
(2), $k = \lfloor \lg n \rfloor$	$2^{k+1} - 1$	$\Theta(n)$	$1 + 2 + 4 = 7$
(3)	$\sum_{i=1}^{n} \lfloor n/i \rfloor$	$\Theta(n \lg n)$	$6 + 3 + 2 + 1 + 1 + 1 = 14$

(1) Sum i operations per iteration, or use the latter half for a quadratic lower bound. (2) The last term 2^k lies between $n/2$ and n, and the total is less than twice that term. A logarithmic number of rounds does not make each round linear. (3) Each iteration visits multiples of i, hence the floor. The harmonic comparison is proved next.

A01-E11 Bound the harmonic sum on both sides

On $[i, i+1]$, $1/x \le 1/i$. Integrating and summing for $i = 1..n$ gives $\int_1^{n+1} dx/x \le H_n$.

On $[i-1, i]$, $1/x \ge 1/i$. Summing for $i = 2..n$ gives $H_n - 1 \le \int_1^n dx/x$. Together: $\ln(n+1) \le H_n \le 1 + \ln n$. For $n \ge 3$, $\ln n \le H_n \le 2 \ln n$. A fixed change of base gives $\Theta(\lg n)$.

For $1 \le x < 2$, $\lfloor x \rfloor = 1 \ge x/2$. For $x \ge 2$, $\lfloor x \rfloor > x - 1 \ge x/2$. Apply this with $x = n/i$ to obtain $nH_n/2 \le \sum \lfloor n/i \rfloor \le nH_n$. These are general bounds, not estimates from finitely many inputs.

A01-E12 Bound a sum without its closed form

Fix $p \geq 0$. Each term is at most n^p, so the sum is at most n^{p+1}. The interval $\lceil n/2 \rceil..n$ contains at least $n/2$ terms, each at least $(n/2)^p$. The sum is therefore at least $n^{p+1}/2^{p+1}$.

Choose $c_1 = 2^{-(p+1)}$, $c_2 = 1$, $n_0 = 1$. Fixing p keeps these factors independent of n. At $p = -1$, the terms decrease, so the claimed lower bound on each latter-half term has the wrong direction. The actual sum is harmonic, $\Theta(\lg n)$ rather than $\Theta(1)$.

A01-E13 Tight logarithms do not imply tight original values

Since $n! \leq n^n$, $\lg(n!) \leq n \lg n$. At least $n/2$ latter-half factors are at least $n/2$, so $\lg(n!) \geq (n/2)\lg(n/2) \geq (n/4)\lg n$ for $n \geq 4$. Constants $1/4, 1$ establish the tight logarithmic bound.

In $n!/n^n = \prod_{i=1}^{n}(i/n)$, the first $\lfloor n/2 \rfloor$ factors are at most $1/2$; every other factor is at most 1. Thus

$$0 \leq \frac{n!}{n^n} \leq 2^{-\lfloor n/2 \rfloor} \longrightarrow 0.$$

Hence $n! = o(n^n)$. A fixed-factor bound between logarithms does not imply a fixed-factor bound between the original values, so the conclusions do not conflict.

A01-E14 Numerical size and machine overflow

$N = 13$ visits $1, 2, 4, 8$, four values; $N = 0$ visits none. For positive N, the number of visits is exactly $b = \lfloor \lg N \rfloor + 1$.

```
if N == 0: return
i = 1
while true
    visit(i)
    if i > floor(N/2): break
    i = 2*i
```

Before updating, $i \leq \lfloor N/2 \rfloor$, so $2i \leq N$ and the result fits the same unsigned word. If this condition fails, the next power of two exceeds N, so execution should stop. Every update still doubles i, ensuring termination.

Enumerating from 1 through N makes N visits. A b-bit positive integer satisfies $2^{b-1} \leq N < 2^b$, so this is $\Theta(2^b)$ visits. This compares iteration counts without treating arbitrary-precision arithmetic as free.

A01-E15 Cumulative allocation versus peak space

Cumulative allocation is $1 + 2 + \cdots + n = n(n+1)/2 = \Theta(n^2)$. With immediate release, only one block is live at a time; its maximum size is n. Peak auxiliary space is $\Theta(n)$.

Retaining all blocks makes them simultaneously live after the last allocation, so the peak becomes $n(n+1)/2 = \Theta(n^2)$. Cumulative allocation is unchanged. These conclusions use the stated reuse and fragmentation assumptions; they are not measurements of an allocator's resident memory.

A01-E16 Refine the analysis using input structure

The first $n-1$ records, $2..n$, are sorted. Each forms one inversion with the final 1, so $I(S) = n-1$. The first $n-2$ iterations each make one failed comparison and no shift. The final iteration shifts every preceding record and stops at the boundary.

Count	General result
Shifts	$n-1$
Key comparisons	$(n-2) + (n-1) = 2n-3$
Array writes	$n-1$ shifts plus $n-1$ final writes: $2n-2$
Time	$\Theta(n + I(S)) = \Theta(n)$

At $n = 2$, the zero early iterations still fit: one comparison and two writes. A quadratic worst case takes a maximum over each length; it does not require every input to be quadratic. For a family with $I(S) \le cn$ and fixed c, $T(S) = \Theta(n + I(S)) = \Theta(n)$, with constants allowed to depend on c, not on n.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A01-E17 Attach error bounds to approximations

The general bound is $e^{1-1/(2n)} \le (1+1/n)^n \le e$. At $n=10$, $e^{0.95} \le 1.1^{10} \le e$. The three values are approximately $2.585710, 2.593742, 2.718282$, respectively, confirming the particular example.

The sharper Stirling bound gives $1/73 < \alpha_6 < 1/72$, hence $Le^{1/73} < 6! < Le^{1/72}$. With $L \approx 710.078185$, the endpoints are approximately 719.872212 and 720.009187, and straddle the exact value, 720.

For a conservative three-decimal interval, round the lower endpoint down and the upper endpoint up: $719.872 < 720 < 720.010$. Rounding an upper endpoint down does not in general preserve the original upper-bound guarantee.

A01-E18 Integer division and a safe implementation

Successive floors give $17 \to 5 \to 2 = \lfloor 17/6 \rfloor$; successive ceilings give $17 \to 6 \to 3 = \lceil 17/6 \rceil$. For $x=1, a=b=2$, however, an inner ceiling followed by an outer floor gives $1 \to 1 \to 0$, whereas $\lceil 1/4 \rceil = 1$. The operations cannot be mixed indiscriminately.

Set $q = \lfloor u/v \rfloor, r = u \bmod v$. Then $\lceil u/v \rceil = q + \mathbf{1}_{r>0}$. If $r=0$, the quotient is integral; otherwise adding 1 reaches the next integer.

Since $v \ge 1$, $q \le u$. When the addition is needed, $qv < u$, hence $q < u$, so adding 1 still gives at most u. If u fits an unsigned word, this addition fits too, without first computing the possibly overflowing $u + v - 1$.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study